

PRACTICAL : WEEK 7

Task - 1:

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;
static int static_var = 20;

void code_function()
{
    printf("Code address    : %p\n", (void *)code_function);
}

int main()
{
    int stack_var = 30;
    int *heap_var = (int *)malloc(sizeof(int));

    *heap_var = 40;

    printf("Memory Addresses:\n");

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address  : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week7
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week7$ nano memory.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week7$ gcc memory.c -o memory
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week7$ ./memory
Memory Addresses:
Code address    : 0x600e408ca1c9
Global address  : 0x600e408cd010
Static address  : 0x600e408cd014
Heap address    : 0x600e58129010
Stack address   : 0x7ffc4167cfbc
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week7$ █
```

OBSERVATION:

The program successfully displayed the memory addresses of the code, global, static, heap, and stack variables. The experiment demonstrated that a Linux process is divided into different memory segments, with each segment serving a specific purpose. The code segment stores program instructions, global and static variables are stored in their respective data areas, dynamically allocated memory is stored in the heap, and local variables are stored in the stack. The addresses showed that these variables occupy different regions of the process's virtual address space. Thus, the memory layout of a Linux process and the organization of its major memory segments were successfully analyzed.

Task - 2:

Make use of the /proc/<pid>/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int global_var = 100;
static int static_var = 200;

int main()
{
    int stack_var = 300;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 400;

    printf("Process is running...\n");
    printf("PID: %d\n", getpid());

    printf("Global variable address: %p\n", (void *)&global_var);
    printf("Static variable address: %p\n", (void *)&static_var);
    printf("Heap variable address: %p\n", (void *)heap_var);
    printf("Stack variable address: %p\n", (void *)&stack_var);

    printf("\nProcess will remain active for 60 seconds.\n");

    sleep(60);

    free(heap_var);

    return 0;
}
```

OUTPUT:

```
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ nano memory.c
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ nano memory_map.c
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ gcc memory_map.c -o memory_map
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ ./memory_map
Process is running...
PID: 1270
Global variable address: 0x569edc007010
Static variable address: 0x569edc007014
Heap variable address: 0x569efe009010
Stack variable address: 0x7ffeaa4e3b1c

Process will remain active for 60 seconds.
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ ./memory_map
Process is running...
PID: 1347
Global variable address: 0x5e365d650010
Static variable address: 0x5e365d650014
Heap variable address: 0x5e366f91d010
Stack variable address: 0x7fff657bba5c

Process will remain active for 60 seconds.
```

```
ananya@Ananya: ~$ cd /mnt/d/OSSP_Programs/Practical/Week7
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ cat /proc/1270/maps
569edc003000-569edc004000 r--p 00000000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
569edc004000-569edc005000 r-xp 00001000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
569edc005000-569edc006000 r--p 00002000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
569edc006000-569edc007000 r--p 00002000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
569edc007000-569edc008000 rw-p 00003000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
569efe009000-569efe02a000 rw-p 00000000 00:00 0 [heap]
71a5c5800000-71a5c5828000 r--p 00000000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
71a5c5828000-71a5c59c0000 r-xp 00028000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
71a5c59c0000-71a5c5a0e000 r--p 001c0000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
71a5c5a0e000-71a5c5a12000 r--p 0020d000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
71a5c5a12000-71a5c5a14000 rw-p 00211000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
71a5c5a14000-71a5c5a21000 rw-p 00000000 00:00 0
71a5c5bea000-71a5c5bed000 rw-p 00000000 00:00 0
71a5c5bf2000-71a5c5bf4000 rw-p 00000000 00:00 0
71a5c5bf4000-71a5c5bf8000 r--p 00000000 00:00 0 [vvar]
71a5c5bf8000-71a5c5bfa000 r--p 00000000 00:00 0 [vvar_vclock]
71a5c5bfa000-71a5c5bfc000 r-xp 00000000 00:00 0 [vdso]
71a5c5bfc000-71a5c5bfd000 r--p 00000000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
71a5c5bfd000-71a5c5c2c000 r-xp 00001000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
71a5c5c2c000-71a5c5c37000 r--p 00030000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
71a5c5c37000-71a5c5c39000 r--p 0003b000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
71a5c5c39000-71a5c5c3a000 rw-p 0003d000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
71a5c5c3a000-71a5c5c3b000 rw-p 00000000 00:00 0
7ffeaa4c4000-7ffeaa4e5000 rw-p 00000000 00:00 0 [stack]
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ cat /proc/1270/maps | grep heap
cat: /proc/1270/maps: No such file or directory
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ cat /proc/1270/maps | grep heap
cat: /proc/1270/maps: No such file or directory
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ cat /proc/1347/maps | grep heap
5e366f91d000-5e366f93e000 rw-p 00000000 00:00 0 [heap]
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ cat /proc/1347/maps | grep stack
7fff6579c000-7fff657bd000 rw-p 00000000 00:00 0 [stack]
ananya@Ananya: /mnt/d/OSSP_Programs/Practical/Week7$ cat /proc/1347/maps | grep memory_map
5e365d64c000-5e365d64d000 r--p 00000000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
5e365d64d000-5e365d64e000 r-xp 00001000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
5e365d64e000-5e365d64f000 r--p 00002000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
5e365d64f000-5e365d650000 r--p 00002000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
5e365d650000-5e365d651000 rw-p 00003000 00:47 2533274790396140 /mnt/d/OSSP_Programs/Practical/Week7/memory_map
```

```

ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week7$ pamp 1347
ctlfile set to 1347.
PAMP in paml version 4.9j, October 2019
ns = 9      ls = 888
Reading sequences, sequential format..
Reading seq # 9: lemur
Sequences read..
Counting site patterns.. 0:00
Compressing, 357 patterns at 888 / 888 sites (100.0%), 0:00
Collecting fpatt[] & pose[], 357 patterns at 888 / 888 sites (100.0%), 0:00
Counting frequencies..

TREE # 1
(((((((1, 2), 3), 4), 5), 6), 7), 8, 9);

Hartigan reconstruction done..
pattern done..
gamma done..
Yang reconstruction done..

```

OBSERVATION:

The program successfully analyzed the virtual memory organization of a running Linux process using the `/proc/<PID>/maps` file and the pmap memory analysis tool. The memory mappings showed different regions of the process, including the executable code, data, heap, stack, and shared libraries, along with their corresponding access permissions. The heap and stack regions were identified using the `[heap]` and `[stack]` mappings. The experiment demonstrated that each process has its own virtual address space, which is divided into multiple memory regions and mapped by the operating system. Thus, the virtual memory mappings and memory organization of a running Linux process were successfully studied.